

Operation Cross-Fire: Protecting the Orbital Corridor



Scenario

Rogue drones, projectiles, and cosmic debris are descending through the orbital defense grid. Two players must operate one shared interceptor and survive a 60-second round.

One player is the Pilot, controlling movement and Boost. The other is the Gunner, controlling aim, fire, and Shield. A Quantum Flux event swaps their roles and increases the difficulty at 20 and 40 seconds.

Evaluation notes

- Engineering over art: Basic shapes, simple sprites, and default UI are sufficient. Do not spend significant time on detailed art, audio, animations, or menus.

- Time box: Do not spend more than 6-8 hours on the assignment. Document any unfinished work. Our goal is to evaluate your velocity and quality.
- Interview defense: Be ready to explain the architecture, input ownership, role switching, pooling lifecycle, and every line of submitted code—including AI-generated code—and make a small live change.
- Scope: This is a local, same-device co-op. Networking is not required.

Game layout and rules

- Landscape 2D playfield with threats spawning at the top and moving downward.
- Exactly one shared ship, initially positioned near the bottom centre.
- The ship moves horizontally and remains inside the playfield.
- The ship starts with three hull points.
- An unshielded enemy, debris, or enemy-projectile collision removes one hull point.
- After taking damage, apply a short configurable invulnerability period so one collision cannot remove multiple hull points.
- The players win when the timer reaches 60 seconds with at least one hull point remaining.
- The players lose when the hull reaches zero or a breach hazard reaches the bottom.
- Stop spawning and gameplay input after the round ends.

Roles and controls

Roles identify the current responsibilities; Player 1 and Player 2 identify the people.

Role	Responsibilities	Unity Editor input
Pilot	Move left/right; activate Boost	A/D or arrows; Space
Gunner	Aim; fire; activate Shield	Mouse movement; left click; right click

Keyboard and mouse are development controls. The physical-device build must provide equivalent touch controls.

Mobile touch controls

- Player 1 uses the left side; Player 2 uses the right side.
- The current Pilot receives Left, Right, and Boost controls.
- The current Gunner receives an aiming area plus Fire and Shield controls.
- Holding Left or Right moves continuously; holding both stops movement.
- Dragging in the aiming area moves the reticle; holding Fire shoots at a fixed cadence.
- Track each touch by ID and keep it assigned to the control where it began until it ends or is cancelled.
- Crossing the centre line must not transfer ownership.
- Shield touches must not also aim or fire.
- Both players' inputs must work simultaneously.

Abilities and weapon defaults

Expose these values as serialized Unity fields:

System	Default behaviour
Boost	Pilot only; 1s duration; 1.75× movement speed; 4s cooldown; no invulnerability
Shield	Gunner only; 1.5s duration; 5s cooldown; prevents hull damage while active
Weapon	Aims toward the reticle; 0.25s firing cooldown; rapid clicking cannot bypass it

Show simple Boost and Shield cooldown indicators. A circle or outline around the ship is sufficient for the active Shield.

Quantum Flux role reversal and progression

Quantum Flux is one event—not a separate mechanic from role reversal. At each transition it cancels active input, swaps the roles, advances the difficulty phase, and displays a brief banner.

Time	Phase	Player 1	Player 2	Difficulty
0-20s	Patrol	Pilot	Gunner	Base enemy speed and spawn interval; enemies and debris appear
20-40s	Alert	Gunner	Pilot	Enemy speed

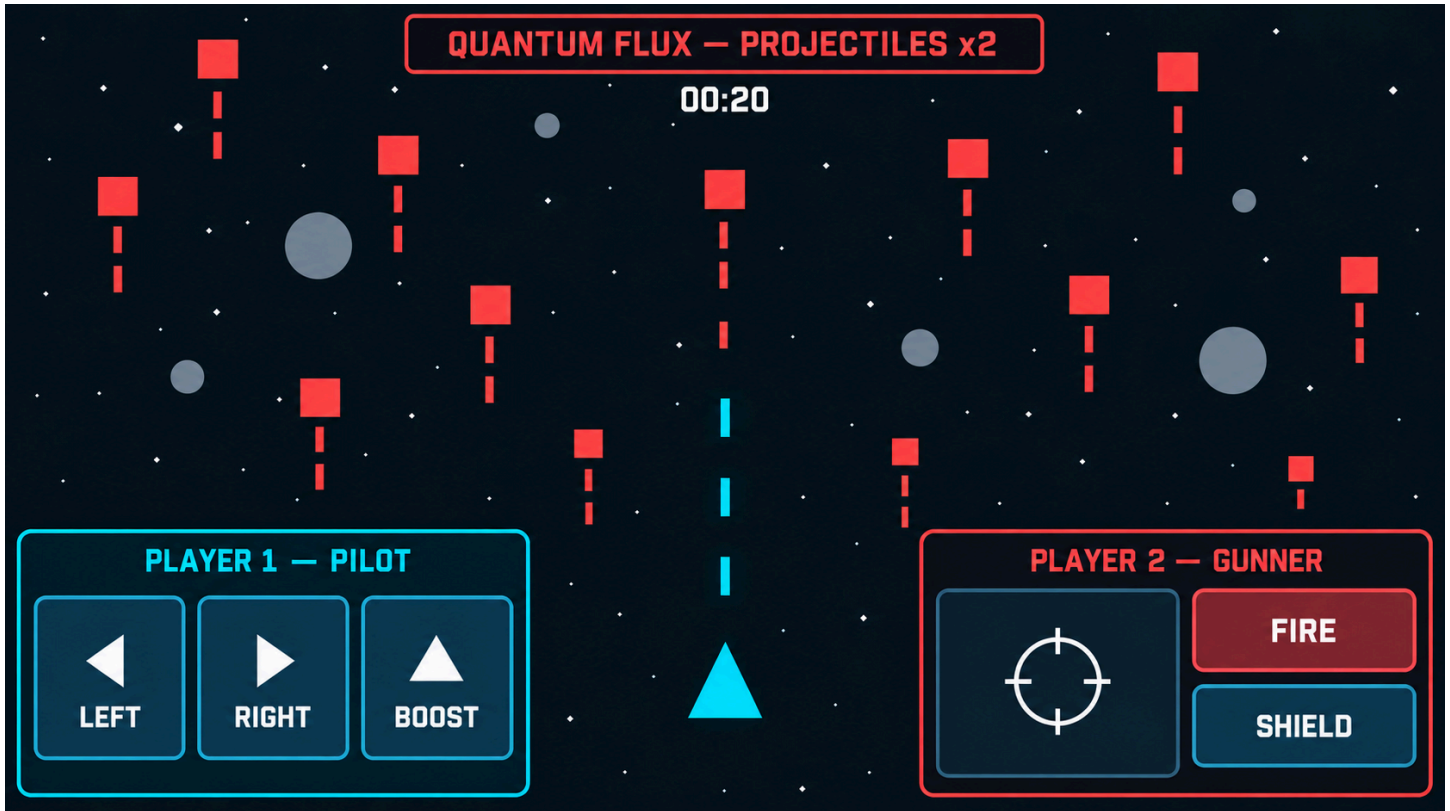
Time	Phase	Player 1	Player 2	Difficulty
				becomes $\text{base} \times 1.25$; breach hazards begin
40–60s	Critical	Pilot	Gunner	Spawn interval becomes $\text{base} \times 0.70$; enemy projectile speed becomes $\text{base} \times 1.50$

Three seconds before a transition, display **QUANTUM FLUX IN 3... 2... 1...**

At the transition:

1. Cancel active movement, firing, Boost, Shield, and touches.
2. Swap Pilot and Gunner assignments.
3. Apply the new phase values.
4. Update the control panels and role labels.
5. Briefly display **QUANTUM FLUX — ROLES REVERSED**.

This should be driven by one round timer and one phase transition—not separate role-swap and Quantum Flux timers.



Gameplay objects and scoring

Object	Behaviour	Health	Score
Enemy	Red square; moves downward; may fire projectiles	1 hit	10
Debris	Grey circle; moves downward; damages ship on contact	2 hits	15
Breach hazard	Orange diamond; starts in Alert; immediate loss if it reaches bottom	3 hits	25
Enemy projectile	Red rectangle; damages unshielded ship	—	—
Player laser	Travels from ship toward reticle	—	—

The HUD must show remaining time, hull, score, phase, current role assignments, and ability cooldowns.

Technical requirements

Collisions

Handle at minimum:

- Player laser against enemies, debris, and breach hazards.
- Enemies, debris, and enemy projectiles against the ship or active Shield.
- Breach hazards against the bottom boundary.
- Spawned objects against an off-screen cleanup boundary.

Use Unity 2D colliders/triggers and configure the physics collision matrix to exclude impossible interactions.

Object pooling

Pool player lasers, enemy projectiles, enemies, debris, and breach hazards. Prewarm the pools before the round begins and reuse objects instead of repeatedly calling `Instantiate` and `Destroy`.

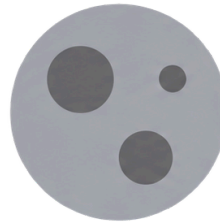
When reused, objects must reset position, rotation, velocity, health, timers, collision state, and visual state. Size pools for the Critical phase; safe expansion is acceptable if documented.

Mobile performance

- Run on one physical Android or iOS device.
- Avoid recurring managed allocations from gameplay code after pool warm-up.
- Avoid repeated scene-wide searches in `Update` or physics callbacks.
- Cache required component references.
- Use the Unity Profiler during the Critical phase and include evidence.

Zero allocation from Unity internals is not required; prevent avoidable recurring allocations in the candidate's gameplay code.

Prototype sprites



The 4x2 reference sheet contains: player ship, enemy, debris, breach hazard, player laser, enemy projectile, Boost icon, and Shield icon. Candidates may use these references, Unity primitives, or equivalent simple sprites.

Acceptance criteria

- A 60-second round can be won or lost using the defined rules.
- Pilot movement and Gunner aiming/firing work simultaneously.
- Boost belongs only to the Pilot; Shield belongs only to the Gunner.
- One Quantum Flux transition occurs at 20 seconds and another at 40 seconds.
- Each transition safely cancels input, reverses roles, and advances difficulty.

- Editor keyboard/mouse and physical-device touch controls work.
- Hull, timer, score, phase, roles, and cooldowns are visible.
- Breach hazards begin during Alert and cause a loss if they escape.
- Frequently spawned objects use pools without repeated active-play instantiation or destruction.
- The game runs smoothly on the selected mobile device.

Out of scope

Networking, accounts, multiple levels, upgrades, alternate ammunition, shield polarity, adaptive difficulty, detailed art/audio/VFX, production menus, tutorials, and save data.

Submission checklist

1. Unity project: Accessible public or private GitHub repository with no credentials, personal data, or non-redistributable assets.
2. Git history: Regular descriptive commits showing meaningful progress.
3. Device evidence: Screenshots and a short physical-device video showing simultaneous controls, Boost, Shield, Quantum Flux, increased difficulty, and a win or loss.
4. Profiler evidence: Capture from the Critical phase after pool warm-up.
5. README: Architecture, input/role switching, pooling/performance, progression, setup/device details, AI usage, assumptions, and incomplete work.